

Workshop 1: Introduction to Pygame Library and Python Game Development

Workshop proposed by
Eng. Meriam Gaied
Under the supervision of Dr. Amur Al-Yahmedi

October 2026

Workshop Objectives

- Get acquainted with the Pygame Python library.
- Calibrate Steering Wheel and Pedals

Workshop Prerequisites

- Python 3.0 Coding Basics.

1 Introduction to Pygame

What is Pygame?

Pygame is a dedicated library to make games in python. It is able to give you full control of the program's execution, which allows for more freedom when coding your program. We will work with the most recent version of Pygame which is Pygame2 released in 2020.

Working With Pygame

It is important to follow the recommended steps when initializing and utilizing the different modules of Pygame given the degrees of freedom it allows when coding a program. So, be careful with your code lines.

When coding with Pygame it is important to start your program by initializing the modules to be used. You can initialize the Pygame module using ***pygame.init()***. Returns: (***numpass, numfail***). What it does is initialize all imported pygame modules. No exceptions will be raised if a module fails, but the total number of successful and failed initializations will be returned as a tuple. You can always initialize individual modules manually, but ***pygame.init()***

initializes all imported pygame modules, which is a convenient way to get everything started. The *init()* functions for individual modules will raise exceptions when they fail.

You may want to initialize the different modules separately to speed up your program or to not use modules your game does not require.

More on Pygame through this link: <https://www.pygame.org/docs/>

2 Calibrating the steering Wheel and Pedals

To check the steering wheel and pedal calibration, as well as detect the buttons, you need to access the game control center.

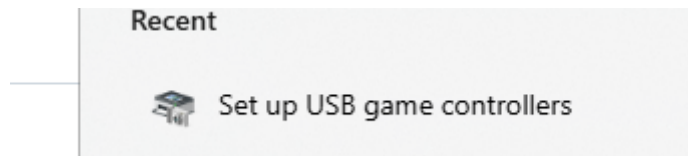


Figure 1: Search Prompt for Game Controller.

If detected, the device will appear in the list of connected devices, and you can access its properties to check for calibration and button detection.

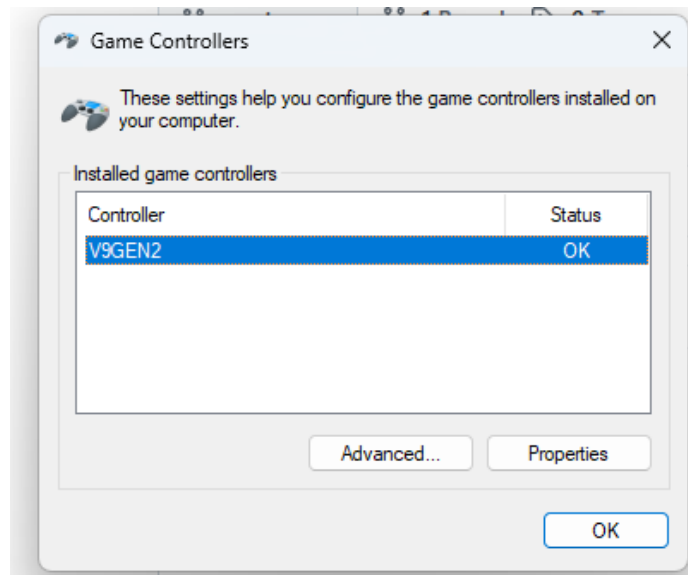


Figure 2: Dialog window for the Game Controller.

Once you consult the properties section of the selected device, you will be able to calibrate it and check the button mapping.

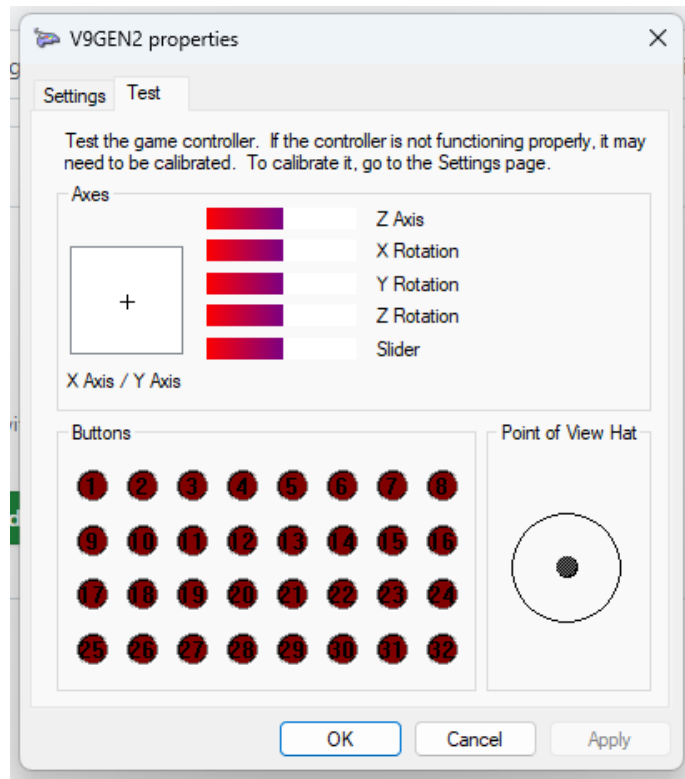


Figure 3: Detection and Calibration Dialog Window.

3 Getting Started with Pygame

In this section, we will code a basic game where we control a square and move it around the Pygame window. We will be using the buttons on the steering wheel to move the square and change its behavior.

Link to demo: <https://github.com/mgaied-sketch/Python-Intro/blob/master/Joysticks>

For that, we will need to consider the controller as a joystick. The following code has gaps that you will need to complete by referencing the documentation of Pygame and the Joystick module.

Code Template

```
import pygame

# Initialize Pygame Module
...
# Initialize Joystick Module
...
# Define Screen Size (Constants)
```

```

...

# Create game window
screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
pygame.display.set_caption("Joysticks")

# Define font
font_size = 30
font = pygame.font.SysFont("Futura", font_size)

# Function for outputting text onto the screen
def draw_text(text, font, text_col, x, y):
    img = font.render(text, True, text_col)
    screen.blit(img, (x, y))

# Create clock for setting game frame rate
clock = pygame.time.Clock()
FPS = 60

# Create empty list to store joysticks
...

# Create player rectangle
x = 350
y = 200
# Definition: Rect(left, top, width, height) -> Rect
player = pygame.Rect(x, y, 100, 100)

# Define player colour
col = "royalblue"

# Define movement speed in number of pixels
speed = 5

# Game loop
run = True
while run:

    clock.tick(FPS)

    # Update background
    screen.fill(pygame.Color("midnightblue"))

    # Draw player
    player.topleft = (x, y)
    pygame.draw.rect(screen, pygame.Color(col), player)

    # Event handler
    for event in pygame.event.get():
        if event.type == pygame.JOYDEVICEADDED:
            joy = pygame.joystick.Joystick(event.device_index)
            joysticks.append(joy)
        # Quit program
        if event.type == pygame.QUIT:
            run = False

# Show number of connected joysticks

```

```

if pygame.joystick.get_count() > 0:
    draw_text("Controllers: " + str(pygame.joystick.get_count()), font, pygame.Color("azure"), 10, 10)
    draw_text("Battery Level: " + str(joy.get_power_level()), font, pygame.Color("azure"), 10, 35)
    draw_text("Controller Type: " + str(joy.get_name()), font, pygame.Color("azure"), 10, 60)
    draw_text("Number of axes: " + str(joy.get_numaxes()), font, pygame.Color("azure"), 10, 85)

# Handle joystick input
for joystick in joysticks:
    # Change player colour with buttons
    ...

    # Player movement with hat (D-pad)
    hat = joystick.get_hat(0)
    ...

    # Keep player within screen boundaries
    if x < 0:
        x = 0
    if x > SCREEN_WIDTH - 100:
        x = SCREEN_WIDTH - 100
    if y < 0:
        y = 0
    if y > SCREEN_HEIGHT - 100:
        y = SCREEN_HEIGHT - 100

#update display
pygame.display.flip()

pygame.quit()

```